BILL-TO-TALLY PROTOTYPE SPECIFICATION

Final implementation blueprint for purchase invoice automation in TallyPrime

Business	Ashan Tech Labs
Deployment	Local Windows computer
Voucher types	Accounting purchase and inventory purchase
Approval flow	Extract → Review → Approve → Post
Test dataset	10–20 sample supplier bills
Tally version	Unknown; connector must be version-configurable

Purpose
Build a local application that reads supplier bills, produces an editable draft, validates accounting data, and creates a TallyPrime purchase voucher only after an authorized user approves it.

Safety rule
No live posting until the installed TallyPrime version, company configuration, voucher format, master mappings, and test-company results have been confirmed.

1. Scope and Workflow

- Input: JPG, PNG, and PDF supplier invoices; single upload and small batch upload.
- OCR: extract supplier name, supplier invoice number/date, GSTIN when present, line descriptions, quantities, rates, discounts, taxes, and totals.
- Review: compare the original document with editable fields; show confidence and warnings; map supplier ledger and stock item/purchase ledger.
- Approval: only approved records can be sent to the Tally connector.
- Posting: record Tally response, created voucher reference where available, and an audit log.
- Exception handling: blurry scan, missing supplier, unmatched stock item, duplicate bill, tax mismatch, Tally offline, import rejection, or ambiguous posting outcome.

2. Architecture

Layer	Technology	Responsibilities
UI	React + TypeScript	Upload, preview, edit, approve/reject, status dashboard
API	FastAPI + Pydantic	Validation, workflow, permissions, integration calls
OCR	PaddleOCR/Tesseract	Image preprocessing and text extraction
Database	SQLite initially; PostgreSQL later	Invoice draft, line items, mappings, posting log
Connector	Tally XML/HTTP adapter	Master lookup, voucher request, response parsing
Files	Restricted local folder	Original invoices and optional OCR text

Tally integration note: TallyPrime supports XML over HTTP. Native JSON integration is documented for Release 7.0 and later. Since the installed release is unknown, implement XML as the initial connector path and keep the transport/payload adapter configurable after version confirmation. Use the exact voucher shape exported from the target company as the template.

3. Suggested Source-Code Structure

```
bill-to-tally/
  backend/
    app/
      main.py                # FastAPI application
      core/config.py         # env vars, paths, Tally endpoint
      core/security.py       # local authentication and roles
      db/session.py          # SQLAlchemy engine/session
      db/models.py           # database models
      schemas/invoice.py     # Pydantic request/response schemas
      api/routes/
        invoices.py          # upload, list, read, edit
        review.py            # validate, approve, reject
        masters.py           # supplier/ledger/item mapping
        tally.py             # health check, masters, post voucher
      services/
        document_store.py    # safe file storage
        image_preprocess.py  # rotate, deskew, contrast
        ocr_service.py       # OCR adapter
        extraction.py        # OCR text -> structured fields
        validation.py        # totals, required fields, duplicates
        mapping_service.py   # supplier/item mapping
        workflow.py          # state transitions
        audit.py             # audit events
      integrations/tally/
        base.py              # connector interface
        xml_client.py        # HTTP XML transport
        xml_templates.py     # voucher request builders
        response_parser.py   # import response parsing
        fake_client.py       # simulation for development
      tests/
        test_validation.py
        test_api_invoices.py
        test_duplicate_guard.py
        test_tally_xml.py
        test_tally_response.py
    requirements.txt
    .env.example
  frontend/
    src/
      pages/Inbox.tsx
      pages/InvoiceReview.tsx
      pages/Settings.tsx
      components/InvoicePreview.tsx
      components/InvoiceFieldsForm.tsx
      components/LineItemsGrid.tsx
      components/ValidationPanel.tsx
      api/client.ts
  data/
    incoming/                # restricted original documents
    processed/
  scripts/
    init_db.py
    import_sample_data.py
  README.md
```

Configuration example

```
APP_ENV=development
DATABASE_URL=sqlite:///./bill_to_tally.db
DOCUMENT_ROOT=./data
TALLY_HOST=127.0.0.1
TALLY_PORT=9000
TALLY_COMPANY_NAME=
TALLY_FORMAT=XML
POSTING_ENABLED=false
```

Keep POSTING_ENABLED=false until test-company checks pass. Do not store credentials or sensitive company data in source control.

4. Database Schema

Use UUIDs or integer primary keys consistently. Store monetary values as Decimal-compatible NUMERIC fields, never binary floating point. Store the original bill path and a SHA-256 file hash.

Table	Key fields	Purpose
invoices	id PK; source_filename; stored_path; file_sha256; status; ocr_text; supplier_name_raw; supplier_ledger_name; invoice_no; invoice_date; currency; subtotal; tax_total; grand_total; tally_company; created_at; updated_at	Invoice header, original file, status, extracted values
invoice_lines	id PK; invoice_id FK; line_no; description_raw; stock_item_name; purchase_ledger_name; quantity; unit; rate; discount; taxable_amount; tax_rate; tax_amount; line_total; confidence	Line items and accounting mapping
master_mappings	id PK; mapping_type; source_text; tally_name; tally_guid nullable; active; verified_at	Supplier, stock item, unit, ledger mappings
approvals	id PK; invoice_id FK; reviewer; decision; comments; created_at	Review and approval history
posting_attempts	id PK; invoice_id FK; idempotency_key UNIQUE; request_hash; started_at; completed_at; status; tally_response; tally_voucher_no; tally_voucher_id; error_message	Posting outcome, retry safety, audit
audit_events	id PK; invoice_id FK; actor; action; before_json; after_json; created_at	Changes and workflow audit trail

Recommended constraints

- UNIQUE posting_attempts.idempotency_key.
- Index invoices(status), invoices(invoice_date), invoices(invoice_no), invoice_lines(invoice_id).
- Do not assume supplier invoice number is globally unique; duplicate detection should compare company + normalized supplier + normalized invoice number and date, then flag for review.
- Do not silently overwrite extracted values when OCR is rerun; retain audit history or create a new extraction revision.

5. API Design

Method / route	Purpose	Expected result
GET /api/health	Check app and database health	Service status
GET /api/tally/health	Check configured Tally endpoint	Reachable/unreachable; never posts
POST /api/invoices	Upload invoice file	invoice_id + queued status
GET /api/invoices	Filter inbox by status/date	Paginated invoice list
GET /api/invoices/{id}	Get extracted header, lines, warnings	Invoice detail
PATCH /api/invoices/{id}	Edit supplier, invoice fields, line items	Updated draft + audit event
POST /api/invoices/{id}/validate	Run required-field, total, tax, duplicate checks	Validation report
POST /api/invoices/{id}/approve	Approve validated draft	Approved state; actor logged
POST /api/invoices/{id}/reject	Reject/send back with reason	Correction-required state
GET /api/tally/masters	Fetch supplier ledgers, purchase ledgers, stock items/units where configured	Mapped master list
POST /api/invoices/{id}/post	Post approved invoice only	Posting attempt and Tally result
GET /api/invoices/{id}/posting	View posting attempts and result	Voucher reference/error

Sample invoice upload response

```
{
  "invoice_id": "inv_01J_SAMPLE",
  "status": "NEEDS_REVIEW",
  "original_filename": "supplier_bill_001.jpg",
  "warnings": ["SUPPLIER_MAPPING_REQUIRED", "LOW_OCR_CONFIDENCE"]
}
```

Sample approve request

```
POST /api/invoices/inv_01J_SAMPLE/approve
{
  "reviewer": "accounts_user_01",
  "comment": "Supplier, quantities and total verified"
}
```

Sample post response

```
{
  "invoice_id": "inv_01J_SAMPLE",
  "status": "POSTED",
  "tally_company": "TEST COMPANY",
  "voucher_type": "Purchase",
  "tally_voucher_number": "PUR/TEST/0001",
  "message": "Tally import accepted; voucher reference reconciled"
}
```

The response above is an illustrative contract, not a real Tally response or a claim that posting has occurred.

6. Sample Test Data

Use synthetic data for development. Replace with real sample invoices only in a controlled test environment. The examples below are not extracted from the blurry bill supplied earlier.

Test ID	Scenario	Sample data	Expected result
INV-T01	Accounting purchase, no tax	Supplier: Demo Stationers; Invoice: DS-1001; Date: 2026-09-15; Purchase ledger: Office Supplies; taxable/base: ■1,000; total: ■1,000	Draft totals reconcile; review required
INV-T02	Inventory purchase with illustrative GST	Supplier: Demo Traders; Invoice: DT-208; Qty: 10; rate: ■100; taxable: ■1,000; illustrative GST total: ■1,180	Inventory lines map; tax setup verified before approval
INV-T03	Blurry image	Low confidence for invoice number and total	Flag for manual correction; posting blocked
INV-T04	Duplicate invoice	Same normalized supplier, invoice number and date as existing draft or posted invoice	Warn/block duplicate pending authorized resolution
INV-T05	Missing supplier ledger	Supplier text has no verified Tally mapping	Require mapping; approval/posting blocked
INV-T06	Tally unavailable	Approved draft; local endpoint not reachable	Mark posting failed/unknown safely; retry after health check
INV-T07	Totals mismatch	Line total and header total differ beyond configured tolerance	Validation error; no post button enabled

Recommended 10–20-bill dataset mix

- 5–8 clean printed invoices across common suppliers.
- 2–4 low-quality photos, rotated scans, or faint print.
- 2–4 invoices with multiple line items and discounts.
- 2–3 invoices with GST details and varying tax structures, checked by an accountant.
- 1–2 duplicate/missing-master/total-mismatch cases.

Use dates, rates, GST rates, tax ledgers, and supplier names from the actual test invoices. Tax values above are synthetic test examples only.

7. Tally Connector Testing Plan

Step 1 — Confirm installed release and company

- Open TallyPrime and identify the exact release from About/Help.
- Create or select a dedicated TEST company; do not start with live books.
- Load the intended test company in TallyPrime before sending requests.
- Enable/configure the local integration HTTP interface as supported by the installed release and confirm the chosen port.
- Keep the endpoint bound to localhost or a trusted private network and verify Windows Firewall rules.

Step 2 — Test read-only connectivity

- Send a harmless export request to confirm Tally is reachable.
- Fetch the active company name and a limited set of ledgers/stock items using a supported export request.
- Compare returned names against the visible Tally company and confirm the connector cannot accidentally target another company.
- Record the exact request, response, release, port, company name, and timestamp.

Step 3 — Validate voucher template

- Manually create one accounting purchase voucher and one inventory purchase voucher in the TEST company.
- Export each voucher to XML (or inspect official sample XML) and use it as the structural reference for the integration adapter.
- Confirm required ledger names, voucher type, date format YYYYMMDD, inventory allocations, units, GST fields, and debit/credit signs.

- Confirm all dependent masters exist: supplier ledger, purchase ledger, tax ledgers, stock item, unit of measure, and godown where applicable.

Step 4 — Controlled posting and reconciliation

- Post one approved accounting purchase and reconcile it in Day Book/Purchase Register.
- Post one approved inventory purchase and confirm quantities, rates, stock values, tax, and supplier balance.
- Parse Tally's response status and import counters; do not rely only on HTTP 200.
- Verify the returned voucher reference and confirm it exists in Tally before marking the app record POSTED.
- Test a timeout after submission: query/reconcile before retrying to prevent duplicate vouchers.

Connector test matrix

Test	Action	Pass condition
TC-01	Tally not running	Health check reports unavailable; no posting attempt
TC-02	Wrong port	Clear connection error; no invoice marked posted
TC-03	Wrong company name	Posting blocked or rejected safely
TC-04	Missing ledger/item master	Readable Tally error; invoice remains unposted
TC-05	Balanced accounting purchase	Accepted; voucher appears in test company
TC-06	Inventory purchase	Voucher and stock quantity reconcile
TC-07	Unbalanced totals	Validation or Tally rejection; no POSTED status
TC-08	Retry same idempotency key	No duplicate voucher
TC-09	HTTP success with import error in payload	Parsed as failure; error visible to user
TC-10	Simulated timeout after submit	Status becomes UNKNOWN/RECONCILIATION_REQUIRED; reconcile before retry

8. Security, Statuses, and Retry Rules

Invoice state machine

```
UPLOADED -> OCR_PROCESSING -> NEEDS_REVIEW
NEEDS_REVIEW -> NEEDS_CORRECTION -> NEEDS_REVIEW
NEEDS_REVIEW -> VALIDATED -> APPROVED -> POSTING
POSTING -> POSTED
POSTING -> POST_FAILED
POSTING -> RECONCILIATION_REQUIRED  # ambiguous timeout/result
```

- Only an authorized reviewer can approve; only an authorized operator/service can post.
- Store original files outside public web roots; restrict file type/size and use generated storage names.
- Do not execute OCR text or invoice content as code. Treat all extracted text as untrusted input.
- Use an idempotency key such as company + normalized supplier + invoice number + date, plus a request hash.
- Never automatically retry a timeout that may have occurred after Tally accepted the voucher; reconcile first.
- Record actor, timestamps, edits, approval, request hash, response body, and Tally voucher identifiers.
- Back up the test company and database before integration testing.

9. Delivery Plan

Days	Tasks	Deliverables
1–2	Setup project, database, upload/storage	App skeleton and schema
3–5	OCR, parsing, synthetic/sample invoice fixtures	Extraction draft
6–8	Review UI, mapping, validation, approvals	Human review workflow
9–12	Tally adapter, master lookup, test voucher imports	Test-company integration
13–15	Test matrix, reconciliation, documentation	Prototype handover and test report

Estimate: 10–15 working days for the first prototype, subject to sample quality, access to a TallyPrime test company, and availability of the correct company voucher templates. This is a planning estimate, not a fixed commitment.

10. Acceptance Criteria

- Upload JPG/PNG/PDF and retain original file with hash.
- Show editable header and line-item fields and warnings for uncertain OCR.
- Support accounting purchase and inventory purchase review paths.
- Block approval when required fields, mappings, totals, or duplicate checks fail.
- Create a voucher only after explicit approval.
- Correctly parse Tally success and error responses.
- Prevent duplicate posting on safe retries; require reconciliation after ambiguous timeouts.
- Show voucher reference or actionable error and retain audit trail.
- Pass all applicable connector test cases in a TEST company before production use.

11. Decisions Required Before Live Posting

- Confirm exact TallyPrime release.
- Confirm company name and whether the target company uses GST and inventory features.
- Confirm purchase voucher type and invoice mode for both accounting and inventory purchases.
- Provide 10–20 sample bills and manually verified expected values.
- Export one sample accounting purchase voucher and one sample inventory purchase voucher from the test company.
- Confirm who can review, approve, and post.

Official integration references

TallyHelp: XML Integration; Integration prerequisites; Sample XML and voucher examples; JSON integration for Release 7.0+.

Status: Final prototype specification. This document does not contain a working Tally connector and no voucher has been posted. Actual voucher XML must be validated against the installed release and the target company's configuration.